

Java Backend Architecture

Interview Series

Chapter 10.5

Relevance Scoring & Boosting

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 10.5 – Relevance Scoring & Boosting

Introduction

Relevance scoring determines the order in which Elasticsearch returns results. The default algorithm, BM25 (Best Match 25), is a probabilistic retrieval model that balances term frequency, inverse document frequency, and document length normalization. Understanding how scores are computed — and how to influence them — is essential for building search experiences where the most relevant results appear first.

Boosting in ES encompasses multiple techniques: query-time field boosting, `function_score` with decay functions for freshness/proximity, pinned queries for editorial control, and `script_score` for fully custom formulas. Choosing the right combination is as much art as science — always validate with an online A/B test against real user queries.

BM25 Score Components

BM25 Score Components

$$\text{score}(q,d) = \text{IDF}(t) \times \text{TF_norm}(t,d)$$

IDF (Inverse Document Frequency)

$$\log(1 + (N - n + 0.5) / (n + 0.5))$$

N = total docs in index

n = docs containing term

Rare terms score higher

TF Normalization

$$\text{freq} / (\text{freq} + k1 * (1 - b + b * dl / \text{avgdl}))$$

freq = term count in doc

k1=1.2 (TF saturation)

b=0.75 (length normalization)

Tuning: increase k1 -> rewards multiple occurrences more

Tuning: decrease b -> reduces length penalty (short docs not over-rewarded)

Customize in index settings: `"similarity": {"default": {"type": "BM25", "k1": 1.5, "b": 0.6}}`

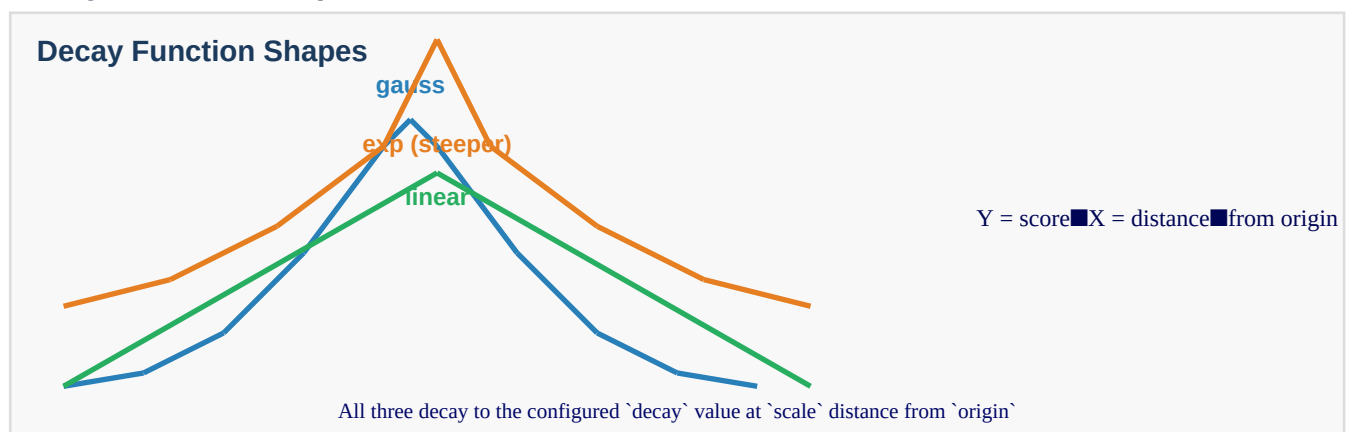
Boosting Strategies

Boosting Strategies

Query-time Boosting	Index-time Boosting	function_score Decay	Pinned Query
^ in multi_match fields - boost param on clause - Affects this query only	- Deprecated in ES 5+ - Stored in norms - Cannot change post-index	- gauss/exp/linear - Recency, distance - Per-query configured	- Pin specific doc IDs - Always rank first - Editorial control

Combine strategies: filter to candidates, function_score for freshness, pinned for editorial picks

Decay Function Shapes



Key Definitions

BM25	Okapi BM25 — default ES similarity; balances TF, IDF, and doc length
TF (Term Freq)	How often a term appears in a document; more occurrences = higher score
IDF (Inv Doc Freq)	How rare a term is across all docs; rarer = higher score
k1	BM25 TF saturation parameter (default 1.2); higher = more reward for repeat terms
b	BM25 length normalization (default 0.75); 0 = no normalization, 1 = full
field norm	Penalty for long fields — short documents with the term rank higher
_score	Computed relevance score for a document; higher = better match
boost	Multiplier applied to a query clause score to increase or decrease its weight
function_score	Query wrapper that modifies scores using custom functions
decay function	Scores decrease as a value moves away from an origin (gauss/exp/linear)
origin	The ideal value for a decay function (e.g., "now" for recency, a lat/lon)

scale	Distance from origin at which the score drops to the decay value
field_value_factor	Multiplies the base score by a document field value (e.g., rating)
pinned query	Forces specific document IDs to appear first regardless of score
script_score	Fully custom scoring via Painless script — most flexible, most expensive
learning to rank	ML-based reranking using features from candidate set (LTR plugin)

BM25 Parameter Tuning Reference

Parameter	Default	Effect of Increasing	Effect of Decreasing	Typical Range
k1	1.2	More reward for additional relevant terms	Score saturates faster (70% on 1.0, 80% on 0.5 less)	0.5 – 2.0
b	0.75	Stronger length normalization	Higher length penalty	0.0 – 1.0

Code Examples

1. Custom BM25 similarity per index

```
// PUT /products { "settings": { "similarity": { "my_bm25": { "type": "BM25", "k1": 1.5, "b": 0.6, "discount_overlaps": true } } }, "mappings": { "properties": { "name": { "type": "text", "similarity": "my_bm25" // Apply custom BM25 to this field }, "description": { "type": "text" // Uses default BM25 (k1=1.2, b=0.75) } } } }
```

2. Query-time field boosting

```
{ "query": { "multi_match": { "query": "wireless headphones", "fields": [ "name^5", // name match worth 5x "tags^3", // tags worth 3x "description^1" // description normal weight ], "type": "best_fields" } } } // Alternatively, boost individual clauses in bool: { "query": { "bool": { "should": [ { "match": { "name": { "query": "headphones", "boost": 5 } } }, { "match": { "description": { "query": "headphones", "boost": 1 } } }, { "term": { "tags": { "value": "wireless", "boost": 3 } } } ] } } }
```

3. function_score — field_value_factor + decay

```
{ "query": { "function_score": { "query": { "bool": { "must": [ { "match": { "name": "running shoes" } } ], "filter": [ { "term": { "in_stock": true } } ] }, "functions": [ { // Boost by rating (sqrt to prevent extreme outliers) "field_value_factor": { "field": "rating", "factor": 1.5, "modifier": "sqrt", "missing": 1.0 }, { // Freshness decay: score drops 50% after 30 days "gauss": { "created_at": { "origin": "now", "scale": "30d", "offset": "7d", // no decay within 7 days "decay": 0.5 } }, { // Proximity boost if user provided location "gauss": { "store_location": { "origin": { "lat": 40.7128, "lon": -74.0060 }, "scale": "5km", "decay": 0.5 } } ] }, "score_mode": "multiply", // multiply all function scores together "boost_mode": "multiply" // multiply combined function score with query score } } }
```

4. Pinned query for editorial control

```
{ "query": { "pinned": { "ids": ["featured-001", "sponsored-042"], // always appear first "organic": { "match": { "name": "running shoes" } } } } } // For dynamic pinning from a database: // 1. Fetch pinned doc IDs from Redis/DB at query time // 2. Inject them into the pinned query // 3. Organic results fill remaining positions // Java: import
```

```
co.elastic.clients.elasticsearch._types.query_dsl.PinnedQuery; PinnedQuery pinnedQuery =
PinnedQuery.of(p -> p .ids(List.of("featured-001", "sponsored-042")) .organic(o -> o.match(m
-> m.field("name").query("running shoes")))) );
```

5. script_score for fully custom scoring

```
{ "query": { "script_score": { "query": { "bool": { "filter": [ { "term": { "category":
"footwear" } } ] } }, "script": { "source": "" // Custom formula: base BM25 * log(1 +
sales_count) * recency_factor double bm25 = _score; double sales = doc["sales_count"].value;
long createdMs = doc["created_at"].value.toEpochMilli(); long nowMs =
System.currentTimeMillis(); double daysSince = (nowMs - createdMs) / 86400000.0; double recency
= Math.exp(-0.05 * daysSince); return bm25 * Math.log1p(sales) * recency; "" } } } } //
WARNING: script_score evaluates for every matching doc – expensive. // Always filter
aggressively before script_score. // Pre-warm with warmers if the filter set is stable.
```

6. Using explain API to debug scores

```
// GET /products/_explain/product-123 { "query": { "match": { "name": "wireless headphones" } } }
// Response: // { // "matched": true, // "explanation": { // "value": 8.123, //
"description": "sum of:", // "details": [ // { "value": 5.2, "description":
"weight(name:wireless ...", "details": [ // { "value": 4.1, "description": "idf, computed as
log(1 + (N-n+0.5)/(n+0.5))" }, // { "value": 1.27, "description": "tf, computed as
freq/(freq+k1*(1-b+b*dl/avgdL))" }, // { "value": 0.99, "description": "fieldNorm" } // ] } // ]
// } // } // Java: add explain=true to a search request SearchRequest req = SearchRequest.of(s
-> s .index("products") .query(q -> q.match(m -> m.field("name").query("wireless")))
.explain(true) ); // Access via hit.explanation()
```

7. A/B testing relevance changes

```
// Pattern: shadow index + score comparison // 1. Create two aliases pointing to different
index versions: // products_a -> products_v1 (old scoring) // products_b -> products_v2 (new
scoring) // 2. Run the same query against both: @Component public class RelevanceABTest {
public void compareRelevance(String query) throws IOException { // Query A
SearchResponse<ProductDocument> respA = esClient.search( s -> s.index("products_a").query(q ->
q.match(m -> m.field("name").query(query))), ProductDocument.class); // Query B
SearchResponse<ProductDocument> respB = esClient.search( s -> s.index("products_b").query(q ->
q.match(m -> m.field("name").query(query))), ProductDocument.class); // Compare rankings,
NDCG, CTR double ndcgA = computeNDCG(respA.hits().hits(), judgments); double ndcgB =
computeNDCG(respB.hits().hits(), judgments); log.info("NDCG A={ } B={ } – delta={ }", ndcgA,
ndcgB, ndcgB - ndcgA); } } // Tools: Quepid (open-source relevance evaluation), RRE (Rated
Ranking Evaluator) // Metrics: NDCG, MRR, Precision@k, ERR
```

Interview Questions & Answers

Q1. How does BM25 differ from TF-IDF?

Both use Term Frequency (TF) and Inverse Document Frequency (IDF) to score documents. The key differences: (1) BM25 applies TF saturation — adding more occurrences of a term beyond a certain point gives diminishing returns (controlled by k_1), while TF-IDF's TF grows unboundedly. (2) BM25 applies document length normalization (b parameter) — long documents are penalized relative to the index average, which TF-IDF doesn't do natively. BM25 is empirically superior and is the default in modern search engines.

Q2. What does the `k1` parameter in BM25 control?

`k1` controls TF saturation: how quickly additional term occurrences stop boosting the score. With `k1=0`, TF doesn't matter at all (only IDF). High `k1` means many occurrences are significantly more valuable than one. Default `k1=1.2` means moderate saturation. For short documents (titles, product names) where each token occurrence is significant, increase `k1`. For long documents (articles) where you expect natural repetition, a lower value prevents frequency gaming.

Q3. What does the `b` parameter control in BM25?

`b` controls document length normalization. `b=1` means full normalization — a term in a 100-word doc scores much higher than the same term in a 10,000-word doc. `b=0` means no length normalization. Default `b=0.75` provides moderate normalization. If your documents vary widely in length and length is meaningful (e.g., a 3-word product title vs. a 500-word description), tune `b`. For uniform-length documents, `b` has less impact.

Q4. What is field norm in ES scoring and can it be disabled?

Field norm encodes the number of tokens in a field at index time as a byte (lossy compression). Longer fields receive a lower norm, acting as a length penalty in BM25. It's stored per-document in a dedicated data structure. Disable with `norms: false` in the field mapping. Useful for keyword-like fields where length doesn't indicate quality (e.g., tags, category names). Disabling norms saves ~1 byte per doc per field and can increase scores for longer instances of those fields.

Q5. Explain the difference between `score_mode` and `boost_mode` in `function_score`.

`score_mode` controls how multiple function scores are combined: `multiply`, `sum`, `avg`, `first`, `max`, `min`. `boost_mode` controls how the combined function score interacts with the query's BM25 score: `multiply` (default), `replace`, `sum`, `avg`, `max`, `min`. Example: two decay functions combined with `score_mode=multiply` (multiply their outputs), then the product multiplied with BM25 using `boost_mode=multiply`.

Q6. What are the three decay function types and how do they differ?

All three decay from `score=1` at the origin to the configured `decay` value at `scale` distance: (1) `gauss` (Gaussian) — smooth bell curve, gradual decay; best for natural quantities like recency where nearby values should score similarly. (2) `exp` (exponential) — steeper drop, closer to origin matters more. (3) `linear` — straight line decay to 0 at `2*scale`; simple but artificially cuts off. Gauss is most commonly used for date/geo proximity boosting.

Q7. When would you use pinned query vs. `function_score` for editorial boosting?

Pinned query is for deterministic, explicitly chosen results that should ALWAYS appear first regardless of the search query — e.g., sponsored listings, featured products, promotional items. The pinned IDs come from a CMS/database and are injected per query. `function_score` is for algorithmic, continuous boosting based on document properties (rating, freshness, proximity) that applies globally. Use both together: pinned for editorial top slots, `function_score` for organic ranking behind them.

Q8. What is the Learning to Rank (LTR) plugin?

LTR is a plugin that enables machine-learning based reranking in ES. You train a model (XGBoost, LambdaMART) on features (BM25 score, recency, CTR, user engagement) and judgments (human-labeled relevance). At query time, ES retrieves `N` candidates with standard query, then rescores them using the trained model. LTR typically dramatically improves NDCG over hand-tuned rules. Requires: feature engineering, offline training, model upload to ES, and rescore query configuration.

Q9. How does the rescore API work?

Rescoring reranks the top `N` documents from a main query using a more expensive secondary query. The main query runs first (cheap, broad), then the rescore query runs only on the top-`N` docs: `rescore: {window_size: 100, query: {rescore_query: {match_phrase: {name: ...}}, query_weight: 0.7,`

`rescore_query_weight: 1.2}}`. Weights control how original and rescore scores combine. Use for expensive queries (phrase, script) that would be too slow as the primary query.

Q10. What is the ``boosting`` query type?

The ``boosting`` query has a ``positive`` clause (must match) and a ``negative`` clause (presence reduces score by ``negative_boost`` factor). Unlike ``must_not``, negative-matching documents are NOT excluded — their score is multiplied by ``negative_boost`` (0-1). Example: search for 'coffee' but demote results containing 'instant': `positive=coffee, negative=instant coffee, negative_boost=0.5`. Real instant coffee results appear, but ranked lower.

Q11. How do you normalize scores across different indices in a cross-index search?

BM25 IDF is computed per-shard, not globally — different shards can produce different IDF values for the same term, causing score inconsistencies in cross-index searches. Solutions: (1) Use ``search_type=dfs_query_then_fetch`` — computes global term/doc statistics across all shards before scoring (slower but consistent). (2) Use a large enough shard with many documents so IDF variance is small. (3) Use ``indices_boost`` to normalize index-level importance.

Q12. What is ``dis_max`` scoring and how does ``tie_breaker`` affect it?

``dis_max`` takes the maximum score from its sub-queries. If ``tie_breaker=0`` (default), only the best field's score counts. ``tie_breaker`` (0-1) adds ``tie_breaker * other_scores`` to the maximum score. With `tie_breaker=0.3`: `final = max_score + 0.3 * (sum of other scores)`. This rewards documents that match in multiple fields without fully summing them. ``multi_match best_fields`` uses `dis_max` internally with the same `tie_breaker` behavior.

Q13. How would you implement a popularity-weighted search (trending products)?

Combine BM25 relevance with a popularity signal using `function_score: field_value_factor` with the ``sales_count`` or ``view_count`` field (using `log1p` modifier to dampen extreme outliers). Additionally, apply a gauss decay on ``trending_since_date`` to reward recently popular items. Time-decay ensures a product that was popular last year doesn't permanently dominate. Expose a toggle that lets users sort by 'relevance' vs. 'popularity' by switching `score_mode`.

Q14. What are the NDCG, MRR, and Precision@k relevance metrics?

NDCG (Normalized Discounted Cumulative Gain): measures the quality of ranking using graded relevance labels; penalizes relevant results appearing lower. Higher NDCG = better ranking. MRR (Mean Reciprocal Rank): measures how high the first relevant result appears; `MRR=1` if it's first, 0.5 if second. Best for navigational queries. Precision@k: fraction of top-k results that are relevant; doesn't account for position within k. Use NDCG for comprehensive evaluation; MRR for 'find the answer' queries.

Q15. How does ``field_value_factor`` handle missing values?

If a document is missing the specified field and no ``missing`` value is configured, ES throws an error at query time. Always set ``missing`` to a sensible default (e.g., 1.0 for multiplicative scoring, 0 for additive). The ``modifier`` transforms the field value: ``none`` (raw), ``log``, ``log1p`` (`log(1+x)` — safe for `x=0`), ``log2p``, ``ln``, ``square``, ``sqrt``, ``reciprocal``. Use ``log1p`` for count fields to prevent extreme outliers from dominating.

Q16. What is ``constant_score`` boosting and when is it useful?

``constant_score`` assigns a fixed ``boost`` score to all matching documents. Use cases: (1) Force a tier of results (premium members always score 10, free members 5). (2) Combine with bool query to ensure certain document types always rank above others regardless of text match. (3) In A/B tests, apply different constant boosts to different buckets. Since it wraps a filter, it's cached — more efficient than `function_score` for simple tiered ranking.

Q17. What is the ``random_score`` function and what is it used for?

``random_score`` in `function_score` assigns a random but deterministic score based on a seed and a field. With the same seed, the same documents get the same scores — so results are random-appearing but consistent within a session. Use cases: (1) Display exploration/discovery feeds (different users see different orders). (2) Load-balance traffic across similar products. (3) Testing infrastructure — reproducible random queries. Set seed per user session ID for per-user consistent randomness.

Q18. How do you boost results that exactly match the query term vs. analyzed matches?

Use a `bool/should` with two clauses: one ``match`` (analyzed, normal boost) and one ``term`` on the ``.keyword`` sub-field (exact match, higher boost). Example: query='Nike' on ``name`` field — should match 'Nike Air Max' via analysis AND exactly match `brand='Nike'` via keyword. The exact match clause gets a higher boost, elevating exact-match docs. This pattern also works for phrase prefix vs. full match priority.

Q19. Explain index-time boosting and why it was deprecated.

Index-time boosting set a per-document ``_boost`` value stored in norms, influencing all queries against those documents. It was deprecated in ES 5 because: (1) it was stored lossy in 1 byte, (2) changing a document's boost required reindexing, (3) it interfered with BM25's length normalization (both share the norms byte). The modern equivalent is a dedicated numeric field (e.g., ``priority_score``) used with ``field_value_factor`` at query time.

Q20. How do you tune search relevance without full reindexing?

Several levers don't require reindexing: (1) query-time boost changes (just update query logic in code). (2) Adding/updating synonym filters via `updateable synonyms` (`synonyms_path` with `reload` via ``_reload_search_analyzers``). (3) Adding runtime fields. (4) Changing `search_analyzer` (query-time only). (5) Updating pinned query IDs. (6) `function_score` formula changes. (7) Adding new fields to mapping and populating them via `update_by_query`. Full reindexing is only needed for changing analyzers applied at index time or changing field types.

Q21. What is the ``_shards.failed`` response field and what causes it?

When an ES query fails on some shards but succeeds on others, ES returns partial results with ``_shards.failed > 0``. Common causes: shard unavailable (node down, rebalancing), query timeout, shard-level exception (OOM, corrupt segment). ES returns the available results rather than failing entirely. In production, always check ``_shards.failed`` and log failures. For consistency-critical queries, set ``allow_partial_search_results: false`` to fail the entire request if any shard fails.